

Technical Section

Surface remeshing with preservation of sharp features through iterative identification and optimization of sample points[☆]Jingjing Liu^a, Yuyou Yao^b, Yue Fei^a, Gaofeng Zhang^a, Liping Zheng^{a,*}^a School of Computer Science and Information Engineering, Hefei University of Technology, Hefei 230601, China^b School of Public Security and Emergency Management, Anhui University of Science & Technology, Hefei 231131, China

ARTICLE INFO

Keywords:

Sample point projection
Edge continuity
Feature preservation
Surface remeshing

ABSTRACT

High-quality triangular meshes with sharp features are necessary for geometric processing applications, e.g., CAD models. Existing approaches either directly utilize the original sharp features as outputs, or predetermine feature sample points and reconstruct feature edges after an optimization process. However, these may result in poor-quality triangular facets close to feature edges or even feature preservation failure. With this regard, we present a feature-preservation surface remeshing method to generate high-quality triangular meshes with sharp feature preservation from raw manifold meshes. Unlike existing predetermination strategy, we introduce a heuristic strategy to dynamically determine feature sample points in each iteration. Based on the projection distance, each feature sample point is identified and projected onto the original feature edges, achieving the preservation of feature sample points. Besides, to eliminate the large gaps between two adjacent feature sample points and edge-close sample points, a feature edge continuity preservation strategy is proposed. Some additional sample points are further projected onto the large gaps and the edge-close sample points are deviated from the feature edges, achieving the preservation of reconstructed feature edges. Experimental results on several models demonstrate the effectiveness and efficiency of our method in generating high-quality triangular meshes with sharp feature preservation.

1. Introduction

Triangular meshes of high quality are frequently used for the representation of three-dimensional (3D) models, which are essential for geometric applications, such as physical simulation and deformation [1], etc. Previous work generates raw meshes by 3D software [2], or reconstructs meshes from sample points collected by 3D scanners [3]. However, these raw meshes suffer from poor quality due to small angles [4], short edges [5], etc. To generate high-quality triangular meshes from these raw meshes, surface remeshing techniques [6–9] have received extensive attention over the last two decades, where the geometric fidelity of the output mesh is a critical criterion, especially for sharp features in CAD models.

Feature preservation plays a critical role in surface remeshing, which aims to preserve sharp features of the original surface, such as feature points, feature edges, etc., in the output surface. Some studies [10,11] directly distribute sufficient sample points on predicted feature edges to generate a triangular mesh with sharp feature preservation. However, this may yield poor-quality triangular facets close to feature edges. Other work [6,12] predetermine feature sample points, which are optimized by mesh smoothing techniques. The feature edges

are then reconstructed from these optimized feature sample points. Nevertheless, predetermination of feature sample points is a challenge, and there may be some large gaps between adjacent feature sample points, leading to the failure of feature preservation.

Therefore, we focus on and handle the limitations in previous work. In this paper, we propose a novel feature-preservation surface remeshing method to yield the high-quality feature preserved triangular mesh from a raw manifold mesh, as shown in Fig. 1. Instead of predetermining feature sample points in previous work, we introduce a heuristic strategy to dynamically identify these feature sample points in the iterative optimization process. To be specific, we calculate the projection distance of each sample point from its nearest original feature edge. The sample point with a projection distance lying in a threshold is considered a feature sample point, which is then projected onto the original feature edges, achieving the preservation of feature sample points. However, there may be large gaps between two adjacent feature sample points and some sample points close to the feature edges, which may disrupt the continuity of reconstructed feature edges, as shown in Fig. 6(a). To address it, we further present a feature edge continuity preservation strategy. That is, more additional sample points

[☆] This article was recommended for publication by Prof. A. Gomes.

* Corresponding author.

E-mail address: zhenglp@hfut.edu.cn (L. Zheng).

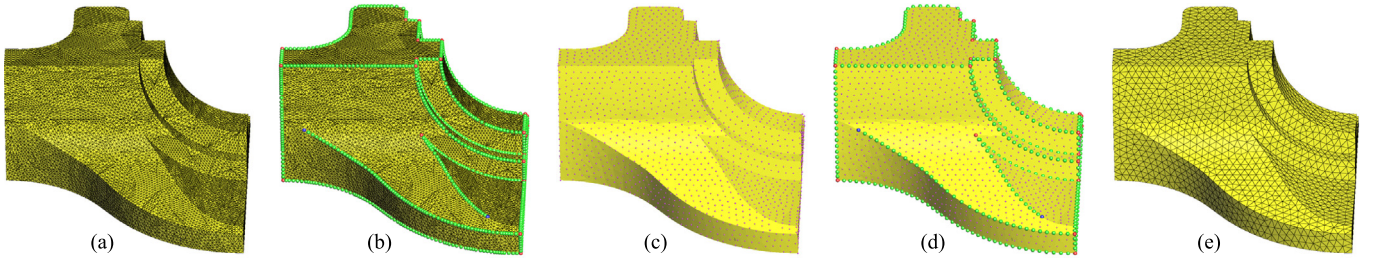


Fig. 1. Overall surface remeshing process of our method with sharp feature preservation. (a) Input model, (b) sharp feature detection, (c) initialized sample points, (d) optimized sample points with sharp feature preservation, and (e) output triangular mesh.

are projected onto the large gaps to eliminate the large gaps between two adjacent feature sample points. Simultaneously, these edge-close sample points are moved away from the feature corner vertices and feature edges to achieve the preservation of reconstructed feature edges. Owing to the independence of the identification and optimization of each feature sample point, the heuristic strategy and feature edge continuity preservation strategy are implemented in parallel on the GPU. We contribute the following:

- We present a feature-preservation method for surface remeshing, where a heuristic strategy is introduced to dynamically identify feature sample points in each iterative optimization process, yielding high-quality triangular meshes with sharp feature preservation.
- We introduce a feature edge continuity preservation strategy, which projects more additional sample points onto the large gaps between two adjacent feature sample points and deviates edge-close sample points away from feature edges, achieving the continuity preservation of reconstructed feature edges.

The remainder of this paper is organized as follows. In Section 2, we briefly review some surface remeshing techniques with feature preservation. Section 3 presents a detailed explanation of our method for feature-preservation surface remeshing. Experimental results on various surfaces are provided in Section 4, and some conclusions are given in Section 5.

2. Related work

feature-preservation remeshing techniques are designed to generate triangular mesh results with sharp feature preservation, such as corner vertices, sharp edges, and geometric shapes in the original mesh. Based on the identification and optimization of feature sample points, existing surface remeshing techniques with feature preservation could be summarized into three categories: predetermination based methods, mesh smoothing based methods, and sample point capture based methods.

Predetermination based methods. Some methods directly distribute a sufficient number of sample points on the detected feature edges of the original mesh. These sample points are not optimized in the subsequent remeshing process, which is directly used for the feature edge construction, achieving the generation of feature-preserved triangular meshes. For instance, based on a user-specified density function, Alliez et al. [13] assign some feature sample points and generalize error diffusion on the original mesh, achieving the preservation of feature edges. Similarly, the original mesh is divided into several smooth patches on the basis of sharp features in [10], and some sample points are uniformly distributed on the edges of these patches to achieve feature preservation. However, this may produce narrow triangular facets in the generated results. Abdelkader et al. [11] present a robust feature-preservation method for surface remeshing, called VoroCrust, which applies pairs of seeds of spherical intersection to capture the sharp features of the original meshes. Nevertheless, the poor-quality of generated triangular facets close to feature edges is also a challenge. In

summary, the predetermination based methods are simple and straightforward, but the sample point optimization and feature preservation are split in the remeshing process, leading to a degradation of the quality of triangular facets close to edges.

Mesh smoothing based methods. There are some approaches that align sample points on the original feature edges by mesh smoothing techniques in the optimization process, thus achieving feature preservation by reconstructing feature edges from these optimized sample points. Valette et al. [14] embed the quadratic error metric (QEM) approximation criterion into the centroidal Voronoi tessellation (CVT) optimization process to solve the sharp feature preservation problem. Later, Yue et al. [15] regularized the triangle geometry by weighted CVT while using local edge operations for improved vertex sampling and imposed constraints on the feature sample points during the optimization process to achieve sharp feature preservation. The main challenge of the method is the trade-off between geometry and connectivity. For instance, to minimize valence, an edge swap operation may be necessary. However, this operation may diminish the geometric improvement achieved in the previous step. ISR [6] detects the feature edges in the local optimization and designs a local operator to preserve sharp features. MAI [12] defines a feature intensity function to implicitly preserve sharp features by classifying and optimizing sample points. The alignment strategy could effectively optimize these sample points located on the feature edges, improving the quality of triangular facets close to the feature edges. CREVO [7] is used to remesh triangular meshes with hard constraints and sharp features. Using novel evolutionary vertex optimization techniques, the algorithm applies to non-differentiable objectives and constraints, such as minimizing the two-sided Hausdorff distance. However, when dealing with intricate mesh models, it is challenging to achieve a balance between preserving features and optimizing triangle quality via mesh smoothing-based methods, which may lead to the loss of sharp features.

Sample point capture based methods. Capturing and reconstructing sample points onto the detected feature edges is the fundamental strategy in these methods. Vorsatz et al. [16] propose a hierarchical curvature field to capture sample points onto the original feature edges, achieving the generation of feature-preserved results. Yan et al. [17] introduce the restricted Voronoi diagram (RVD), which projects the sample points to the original edges located in the corresponding Voronoi cells. A generalized framework is provided in [18] that uses a local smoothing operator to capture sample points to yield triangular mesh with sharp feature preservation. Hurtado et al. [8] proposed a new automatic method to simplify CAD models by computing an adaptive envelope for wrapped CAD models. This envelope is represented by a 2-manifold triangular mesh that iteratively approximates the outer surface of the CAD model to capture its sharp features. However, the sample point capture strategies in different methods cannot be generalized. Besides, capturing sample points onto the feature edges may disrupt the topology structure, resulting in the quality degradation of generated triangular meshes. Our method is also founded on this principle. By selectively attracting sample points in close proximity to the edges of the feature and continuously adjusting the arrangement of these sample points in a dynamic manner, we have effectively alleviated the problem of degradation in triangle mesh quality to some extent.

Algorithm 1: feature-preservation surface remeshing

Input: the raw mesh M , number of sample points n , and feature angle threshold $\theta^\#$

Output: triangular mesh M'

- 1 Detect and extract feature information $G = \{\mathbf{V}^\dagger, \mathbf{E}^\dagger\}$ from the raw mesh M // See Section 3.2
- 2 Sample points $\mathbf{S} = \{\mathbf{s}_i\}_{i=1}^n$ from the raw mesh M
- 3 **do**
- 4 Optimize sample points $\mathbf{S}^\dagger$ by RTF [9]
- 5 Identify and project feature sample points $\mathbf{S}^\dagger \subset \mathbf{S}$ onto $\mathbf{E}^\dagger$ // See Algorithm 2
- 6 Optimize feature sample points $\mathbf{S}^\dagger$ for feature preservation // See Algorithm 3
- 7 Count number of large gaps N_G
- 8 **while** $N_G \neq 0$;
- 9 Extract the triangular mesh M' from optimized $\mathbf{S}$

3. Methodology

The main components of our methodology are (1) sharp feature detection of the original surface (Section 3.2), (2) feature sample point identification (Section 3.3), and (3) sample point optimization and triangular mesh extraction (Section 3.4).

3.1. Overall idea

Given that an input manifold triangular mesh $M = \{\mathbf{V}^m, \mathbf{E}^m, \mathbf{F}^m\}$ consisted of a set of vertices $\mathbf{V}^m = \{v_i^m\}_{i=1}^{n_v^m}$, a set of edges $\mathbf{E}^m = \{e_i^m\}_{i=1}^{n_e^m}$, and a set of facets $\mathbf{F}^m = \{f_i^m\}_{i=1}^{n_f^m}$, where n_v^m , n_e^m , and n_f^m are the number of vertices, edges, and facets. Any edge e_i is formed by two vertices, and any combination of three ordered vertices or edges represents a facet t_i . Let $\mathbf{S} = \{\mathbf{S}^\dagger, \mathbf{S}^\dagger\} = \{s_i\}_{i=1}^n$ denote n distinct sample points, where $\mathbf{S}^\dagger$ and $\mathbf{S}^\dagger$ are the sets of feature sample points and non-feature (or called “ordinary”) sample points, respectively.

In this paper, we propose a novel feature-preservation method for surface remeshing, which aims to generate high-quality triangular mesh with sharp feature preservation from the raw mesh. Given an input mesh M , we firstly traverse each edge e_i and compute the dihedral angle between its neighbouring triangular facets. The sharp feature information $G^\dagger = \{\mathbf{V}^\dagger, \mathbf{E}^\dagger\}$ (e.g., feature vertices $\mathbf{V}^\dagger \subset \mathbf{V}^m$, feature edges $\mathbf{E}^\dagger \subset \mathbf{E}^m$, etc.) is detected and extracted based on the dihedral angle and a predefined threshold $\theta^\#$. The sample points $\mathbf{S}$ are uniformly sampled and initialized from the raw mesh M . Different from the pre-determination strategy of feature sample points in existing techniques, we introduce a heuristic strategy that calculates the distances between each sample point $s_i \in \mathbf{S}$ and the original feature edges $\mathbf{E}^\dagger$ to dynamically identify these feature sample points $\mathbf{S}^\dagger$ in each iteration. Moreover, to generate triangular meshes with high quality and feature preservation, we develop a projection strategy that restricts feature sample points $\mathbf{S}$ onto the original feature edges $\mathbf{E}^\dagger$ and optimizes their positions, achieving the continuity of reconstructed feature edges. The pseudo-code of our method for surface remeshing is provided in Algorithm 1, where N_G counts the large gaps of reconstructed feature edges.

3.2. Sharp feature extraction

The fundamental requirement of our method is to preserve the sharp features of the original mesh. Therefore, it is critical to detect and extract the sharp feature information in the original mesh. Given an input mesh M , the sharp feature information mainly consisted of feature vertices $\mathbf{V}^\dagger \subset \mathbf{V}^m$ and feature edges $\mathbf{E}^\dagger \subset \mathbf{E}^m$. These feature edges $\mathbf{E}^\dagger$ are utilized as the baselines for subsequent feature

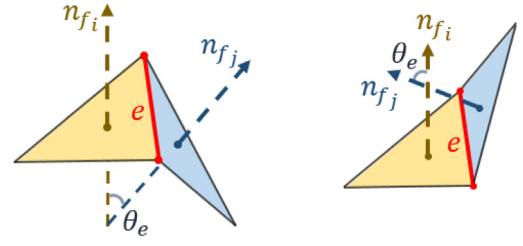


Fig. 2. Compute the angle θ_e formed by the normals of two adjacent facets. n_{f_i} denotes the normal of facet f_i , and n_{f_j} denotes the normal of facet f_j . When $\theta_e \geq \theta^\#$, the common edge e of facet f_i and facet f_j is the feature edge, and it is represented by the red line segment.

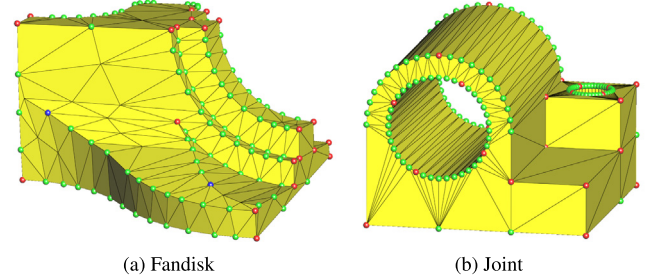


Fig. 3. Sharp feature detection and extraction of the *Fandisk* (left) and *Joint* (right) models, where green dots represent ordinary feature vertices, red dots denote feature corner vertices, and blue dots are feature boundary vertices.

sample point projection, achieving the sharp feature preservation and geometric fidelity of remeshed results, as shown in Fig. 1.

Considering feature edge detection and extraction, we use the edge-based method to identify feature edges by calculating the angle between the normals of adjacent triangular facets. Specifically, we traverse each edge $e \in \mathbf{E}^m$ in the original mesh M and obtain the relevant adjacent triangular facets. Then, based on the coordinates of three ordered vertices, the normals of two adjacent triangular facets could be calculated, which are further fed to calculate the normal angle θ_e . When the normal angle θ_e exceeds a predefined threshold $\theta^\#$, the edge is then considered as a feature edge, that is, $e \in \mathbf{E}^\dagger$, as shown in Fig. 2. It should be mentioned that the threshold $\theta^\#$ is default set to 45° in our work. That is, if the angle formed by the normals of two adjacent facets is 45 degrees or greater, or if the internal angle formed by two adjacent facets is 135 degrees or less, the edge is considered as a feature edge.

In terms of detecting and extracting feature vertices, we could obtain that each feature edge e is composed of a combination of two feature vertices. Therefore, we could directly identify feature vertices $\mathbf{S}^\dagger$ based on the extracted feature edges $\mathbf{E}^\dagger$ of the original mesh M . Then, we count the feature degree $d^f(v_i^\dagger)$ of each feature vertex $v_i^\dagger \in \mathbf{V}^\dagger$ and further classify it into ordinary feature vertices $\mathbf{V}_{ord}^\dagger$, feature corner vertices $\mathbf{V}_{cor}^\dagger$, and feature boundary vertices $\mathbf{V}_{bou}^\dagger$, as shown in Fig. 3. That is,

$$d^f(v_i^\dagger) = \begin{cases} 1, & v_i^\dagger \in \mathbf{V}_{bou}^\dagger \\ 2, & v_i^\dagger \in \mathbf{V}_{ord}^\dagger \\ \geq 3, & v_i^\dagger \in \mathbf{V}_{cor}^\dagger \end{cases} \quad (1)$$

where the feature degree $d^f(v_i^\dagger)$ of a feature vertex $v_i^\dagger$ is defined as the number of feature edges with that vertex as one endpoint, as shown in Fig. 4. Note that these feature corner vertices $\mathbf{V}_{cor}^\dagger$ and feature boundary vertices $\mathbf{V}_{bou}^\dagger$ are directly taken as initialized sample points and not optimized in the subsequent iterative optimization process.

Combining the extracted feature vertices and feature edges, we further divide these feature edges into different groups with unique

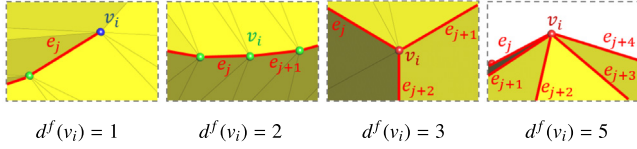


Fig. 4. Classification of feature vertex. The feature vertex $v_i^l \in \mathbf{V}^l$ is classified into feature boundary vertices $\mathbf{V}_{bou}^l$ ($d^f(v_i) = 1$, blue dots), ordinary feature vertices $\mathbf{V}_{ord}^l$ ($d^f(v_i) = 2$, green dots), and feature corner vertices $\mathbf{V}_{cor}^l$ ($d^f(v_i) \geq 3$, red dots) according to the feature degree $d^f(v_i)$.

feature vertices (called “feature index vertex”). This provides a potential requirement for parallel computation of subsequent feature sample point optimization. To be specific, these feature corner vertices and feature boundary vertices are directly considered feature index vertices. These feature edges are then divided into different groups based on their connectivity to the feature index vertices. Each group of feature edges corresponds to a unique feature index vertex, as shown in the *Fandisk* model in Fig. 3. However, there may be some special structures without feature index vertex, e.g., the ring structure of the *Joint* model in Fig. 3. In this case, we designate the first detected feature vertex and its farthest feature vertex on the ring structure as the feature corner vertices. In order to ensure that the feature vertex indices within each group are unique, each half-ring with feature edges will be subdivided by its intermediate feature vertices. Thus, these feature edges on the ring structures are divided into four different groups with corresponding feature index vertices, as shown in the *Joint* model in Fig. 3.

3.3. Feature sample point identification

Based on uniformly initialized sample points $\mathbf{S}$, our method aims to iteratively optimize these sample points. This generates a high-quality triangular mesh that conforms to the sharp features of the original mesh. These sharp features in the generated results are reconstructed from optimized feature sample points $\mathbf{S}^l \subset \mathbf{S}$. Therefore, feature sample point identification is crucial for generating triangular meshes.

Different from the predetermination strategy in previous work [19, 20], we introduce a heuristic strategy to dynamically identify feature sample points $\mathbf{S}^l$ in each iteration. The basic idea is that a smaller distance means that the sample points are closer to the original feature edges and are more important in the feature edge reconstruction of generated triangular meshes. Therefore, we calculate the distance of each sample point s_i from the original feature edges $\mathbf{E}^l$ in each iteration, as shown in Fig. 5. The sample point s_i with the distance d_i satisfied a threshold D_i is considered as the feature sample point, as shown in Fig. 5(a). Note that, the threshold D_i is calculated as:

$$D_i = \delta \cdot \bar{d}_i \quad (2)$$

where $\bar{d}_i$ is calculated as the average distance of the sample point s_i and its m -nearest neighbouring sample points, and the values of δ and m are set to 0.3 and 8, respectively. A point is usually surrounded by 5 to 8 points, and taking any value of m from 5 to 8 will not affect the final result. Thus, we take a value of 8 for m here. The value of δ affects the minimum distance at which the feature edges pull the sample points towards it. The larger the value of δ , the more feature sample points are added in each round of iteration. The convergence of the algorithm is thus faster. Under our extensive experiments, the quickest convergence rate is reached when $\delta \geq 0.3$. In addition, when the value of δ is too large, it will cause overcrowding on the feature edges and reduce the mesh quality. Therefore, we chose $\delta = 0.3$.

The pseudo-code for the feature sample point identification is provided in Algorithm 2. Given the feature information $G = \{\mathbf{V}^l, \mathbf{E}^l\}$ of the original mesh M and sample points $\mathbf{S} = \{s_i\}_{i=1}^n$, we traverse each sample point s_i and search the k nearest neighbouring feature vertices

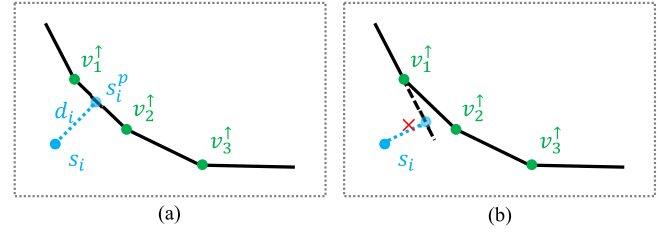


Fig. 5. Exemplification of feature point projection process, where blue dots represent feature sample points, and green dots denote k -nearest-neighbour feature vertices in $\mathbf{V}^l$. (a) Feature sample point s_i is projected on the edge $v_1^l v_2^l$; (b) Sample point s_i is not allowed to be projected onto the extension of the feature edge.

Algorithm 2: Feature sample point identification

Input: extracted feature information $G = \{\mathbf{V}^l, \mathbf{E}^l\}$, sample points $\mathbf{S} = \{s_i\}_{i=1}^n$

Output: Feature sample points $\mathbf{S}^l \subset \mathbf{S}$

```

1 for  $s_i \in \mathbf{S}$  in parallel do
2   Search  $k$ -nearest-neighbour feature vertices  $\mathbf{V}^{k,l}$ 
3   Calculate the threshold  $D_i$  based on Eq. (2)
4   for  $e_j \in \mathbf{E}^l$  ( $\exists v \in \mathbf{V}^{k,l} \ \& \ v \in e_j$ ) do
5     Compute the projected point  $s_{ij}^p$  on  $e_j$ 
6     Calculate the projection distance  $d_{ij}$ 
7     if  $s_{ij}^p \in e_j$  &  $d_{ij} < D_i$  then
8       Identify  $s_i$  as feature sample point:  $\mathbf{S}^l \leftarrow s_i$ 
9     end
10  end
11 end

```

$\mathbf{V}^{k,l}$ from the original feature vertices $\mathbf{V}^l$. The KNN algorithm obtains the nearest k query points of the target points by the magnitude of the Euclidean distance between the target points set and the query points set. The sample point s_i is then projected onto the feature edges that contain neighbouring feature vertices. The projection distance d_i is calculated from the nearest neighbouring projection point s_{ij}^p that is located on the original feature edge. Thus, the sample point s_i is considered as a feature sample point when the projection distance d_i satisfies the calculated threshold D_i in Eq. (2). During this stage, we compute the closest projection distance for each sample point and determine whether to include the sample point in the feature edge based on this distance. This process can be highly parallelized and executed on a GPU.

3.4. Optimization

Based on the heuristic strategy for identifying feature sample points, we classify these sample points $\mathbf{S}$ into feature sample points $\mathbf{S}^l$ and non-feature sample points $\mathbf{S}^l$ (or called “ordinary sample points”), that is, $\mathbf{S} = \mathbf{S}^l \cup \mathbf{S}^l$. Therefore, we further optimize these sample points $\mathbf{S}$ to yield regularly distributed sample points, thus generating a triangular mesh with high quality. Meanwhile, these feature sample points should be located on the original feature edges $\mathbf{E}^l$, achieving feature-preserved results.

Existing techniques have made significant breakthroughs in surface remeshing, e.g. parameterization-based [20] methods and CVT-based [9,21], etc. These methods could be directly utilized for optimizing ordinary sample points in our work. To guarantee the same dimensional of the optimization process, we adopt a 3D CVT-based method, called RTF [9], as the basic strategy for optimizing ordinary sample points $\mathbf{S}^l$ in our work. It calculates an approximating surface for each sampling point on the original surface, replacing the computationally expensive intersection process in RVD [17]. Moreover, the

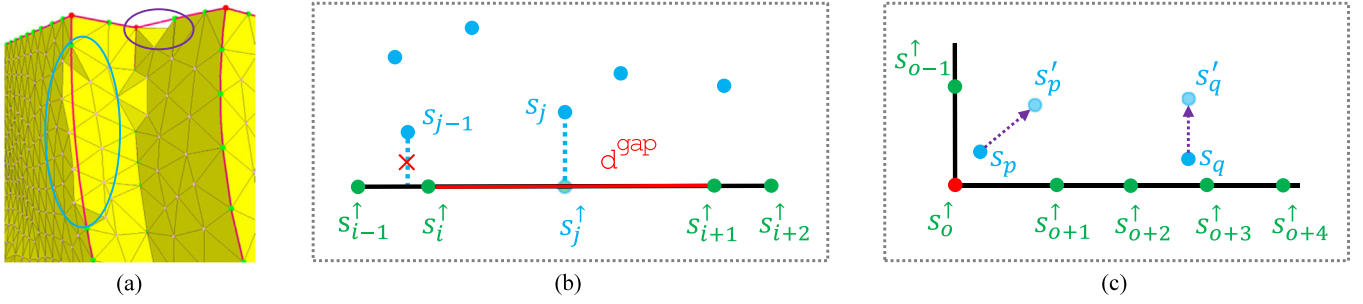


Fig. 6. Exemplification of feature edge continuity preservation in sample point optimization. (a) generated triangular mesh without feature edge continuity preservation caused by large gaps (blue circle) or edge-close sample points (purple circle); (b) projection of ordinary sample point s_j to eliminate large gaps d_i^{gap} ; and (c) ostracism of edge-close sample points s_p and s_q .

Algorithm 3: Feature edge continuity preservation

Input: extracted feature information $G = \{V^\dagger, E^\dagger\}$, sample points $S = \{S^\dagger, S^\downarrow\}$
Output: optimized feature sample points $S^\dagger$

```

1 for  $s_i \in S$  in parallel do
2   if  $s_i \in S^\dagger$  then
3     Calculate the distance  $d_i^{gap}$  of  $s_i$  from its adjacent feature
       sample point
4     if  $d_i^{gap} > D_i^{gap}$  then
5       Search the nearest-neighbour ordinary sample point
6        $s_j \in S^\downarrow$ 
7       Project  $s_j$  onto the original feature edges
8     end
9   else
10    Calculate the distance  $d_i^{near}$  of  $s_i$  from the nearest feature
       edges
11    if  $d_i^{near} < D_i^{near}$  then
12      Compute the position  $s'_i$  based on Eq. (3)
13       $s_i \leftarrow s'_i$ 
14    end
15  end
16 Optimize each feature sample point to the centroid of the
   connected feature edges

```

algorithm steps are independent of each other and can be executed on a GPU. Therefore, our method could produce the regular distribution of ordinary sample points, improving the quality of generated triangular meshes, as shown in Algorithm 1.

In terms of the optimization of feature sample points, we apply the heuristic strategy to identify feature sample points $S^\dagger$, which are further restricted onto the original feature edge $E^\dagger$ based on the projection method. In fact, the identification and projection of feature sample points are performed simultaneously, as shown in Algorithm 2. In the heuristic strategy, we compute the projected point s_i^p of each sample point s_i on the original feature edges $E^\dagger$, and the feature sample point identification is performed according to the projection distance d_i . Therefore, we could directly optimize each feature sample point s_i to the relevant projected point s_i^p to achieve the reconstruction of sharp features. However, there may be some large gaps between two adjacent feature sample points or edge-close sample points, which cause the quality degradation of generated results or even feature preservation failure, as shown in Fig. 6. To address this, we further extend the projection method to achieve the continuity of reconstructed feature edges, as shown in Algorithm 3.

To be specific, considering the large gaps of two adjacent feature sample points, we first traverse each feature sample point $s_i^\dagger$ and calculate the distance d_i^{gap} from its adjacent feature sample point. The

distance d_i^{gap} greater than a threshold $D_i^{gap} = \lambda \cdot \bar{d}_i$ indicates a large gap between two adjacent feature sample points, where λ is default set to 1.4, and $\bar{d}_i$ could be referred to Eq. (2). The value of λ affects the minimum distance at which our algorithm determines that a gap occurs between feature sample points. A smaller value of λ ensures the stability of our method in terms of feature preservation, but this increases the time for the algorithm to converge. With our extensive experiments, when $\lambda \leq 1.4$, features are well preserved in all our experimental models. Therefore, considering the running time and the algorithm's stability, we chose $\lambda = 1.4$. As shown in Fig. 6(b), we search the nearest-neighbour ordinary sample point s_j from the ordinary sample points, which is then projected onto the original feature edges $E^\dagger$ and considered as feature sample point: $S^\dagger \leftarrow s_j$. It should be pointed out that the nearest-neighbour ordinary sample point whose projected point is located outside the large gap is ignored, as shown s_{j-1} in Fig. 6(b). Then, the second-nearest-neighbour ordinary sample point is further taken into consideration, as shown s_j in Fig. 6(b). During this procedure, the distance is calculated between every individual feature sample point. According to our grouping technique, we can rapidly and autonomously identify all neighbouring points of each feature sample point based on the distance using feature index points. The closest sample point is captured onto the gap when the distance is greater than D_i^{gap} . Therefore, our algorithm is applicable to GPUs.

Additionally, in the field of edge-close sample points, we also traverse each ordinary sample point s_p and compute the projected point s_p^p on the original feature edge $e_p \in E^\dagger$. The projection distance d_p^{near} (i.e., $\|s_p - s_p^p\|$) less than a threshold $D_p^{near} = \sigma \cdot |e_p|$ demonstrates an edge-close sample point, where $|e_p|$ is the length of e_p , σ is default set to 0.75. To eliminate the effect of edge-close sample points, we move them away from the feature sample points and reconstructed feature edges. That is, the deviated location s'_p of each edge-close sample point s_p is computed as:

$$s'_p = \begin{cases} s_p + \frac{\sigma \cdot |e_p|}{\|s_p - s_p^p\|} \cdot (s_p - s_p^p), & \forall s \in e, s \notin V_{cor}^\dagger \\ 2s_p^p - v_{cor}^\dagger + \frac{\sigma \cdot |e_p|}{\|s_p - s_p^p\|} \cdot (s_p - s_p^p) \\ \quad + \frac{\|s_p - s_p^p\|}{\sigma \cdot |e_p|} \cdot (v_{cor}^\dagger - s_p^p), & \exists s \in e, s \in V_{cor}^\dagger \end{cases} \quad (3)$$

where $v_{cor}^\dagger \in V^\dagger$, s_p , s_p^p , and $|e_p|$ are the corner vertex of the original mesh M , the sample points, the projection points of the sample points on the feature edge e_p , and the length of the feature edge e_p , respectively. Thus, we optimize each sample point s_p near an edge to its deviated position s'_p , preserving the continuity of the reconstructed feature edges. However, when s belongs to the set of corner vertices, vertically moving s_p may place it within another D_p^{near} region, requiring careful consideration. Therefore we adjust the offset angle when s_p is moved according to the ratio of the projection distance of s_p and the distance from the projected point s_p^p to the corner vertex s . In brief, the shorter the distance from the projected point s_p^p to the corner vertex s , the larger the offset angle. The value of σ determines the distance

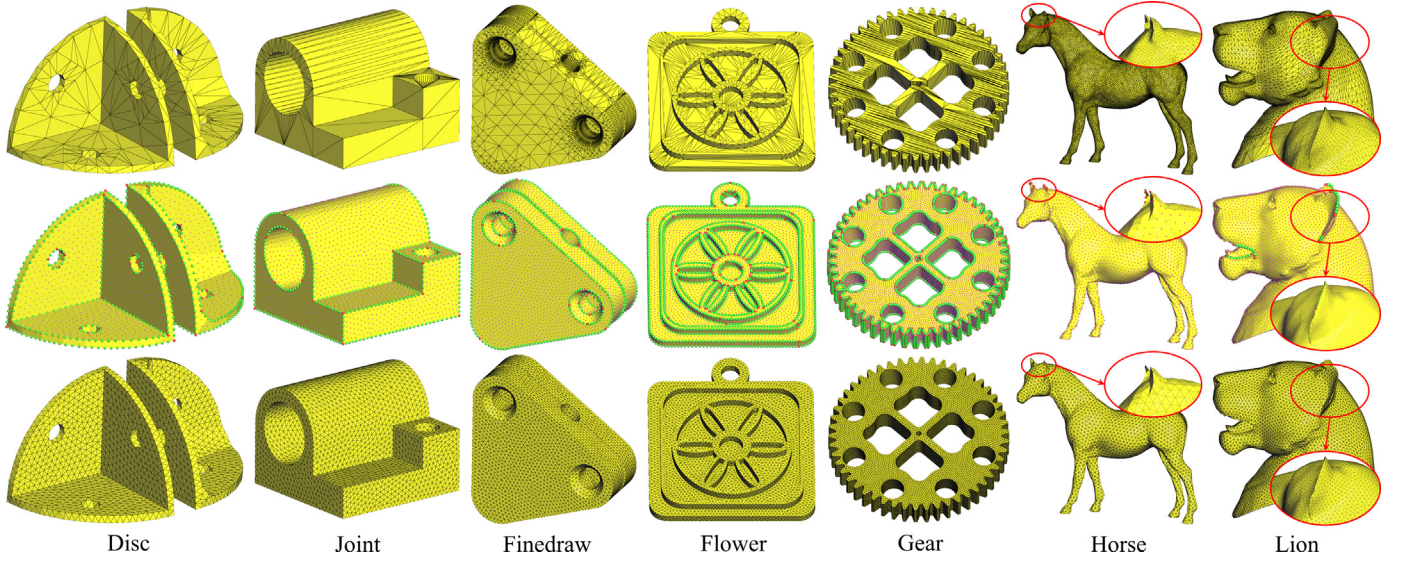


Fig. 7. Feature-preserved remeshing results for seven different models with sharp features. Top: input models. Middle: optimized sample points, where feature sample points are divided into index points (red dots) and feature edge points (green dots). Bottom: output triangular meshes. In our method, feature sample points are optimized using 1-D CVT, while non-feature sample points are optimized using RTF [9].

between the sample points and the edge away from it. Initially, we set the displacement distance to the height of the equilateral triangle with the highest quality, i.e., $\sigma = \sin 60^\circ \approx 0.866$. However, extensive experiments have demonstrated that while ensuring feature preservation, a value of $\sigma = 0.75$ typically generates the highest quality triangles. The possible reason is that the distance moved by the sample points at $\sigma = 0.866$ is too large, which obtains a locally optimal solution but destroys the overall regular structure. Ultimately, to achieve a regular distribution of feature sample points, we compute and optimize each feature sample point to the centroid of the two adjacent reconstructed feature edges, yielding high-quality triangular meshes.

3.5. Mesh generation

After the iterative optimization process described above, a regularly distributed set of sample points can be generated, as depicted in Fig. 1(d). Thus, a triangular mesh should be extracted from these optimized sample points, as shown in Fig. 1(e). There are some existing techniques to extract the desired triangular mesh from sample points, such as Poisson distribution-based method [22,23], reconnection strategy of approximate CVT-based [9,24], and RVD-based method [17, 25]. In this paper, we utilize the robust RVD-based mesh extraction method [17], which computes the constrained Voronoi cells and reconnects the sample points to yield the triangular mesh. Note that the optimized sample points are only used as inputs to generate the desired triangular mesh in the RVD-based mesh extraction method.

4. Experiment

In this section, we demonstrate the effectiveness of our proposed method for surface remeshing with sharp feature preservation and investigate the influence of some parameters. Besides, we also compare our method with the MAI [12], RTF [9], VoroCrust [11] and CREVO [7] to evaluate the feature-preserved remeshing results. All experiments are performed on a Windows 10 computer with 3.6 GHz Intel (R) Core (TM) i7-9700K CPU with 16 GB memory and an NVIDIA GeForce RTX 2080 Ti with 11 GB memory, using CUDA version 10.0.

Parameter settings. The RTF framework in [9] is utilized to optimize these ordinary sample points in each iteration, and the parameters of GPU implementation in this framework could be referred to [9]. Besides, the value of θ is default set to 45° to detect sharp feature

information, and the values of the parameter δ , λ , and σ are set to 0.3, 1.4, and 0.75, respectively.

Mesh quality measurement. We have adopted the evaluation criteria outlined in [26] to assess the quality of the generated meshes in our experimental study. For each triangular facet t , the quality metric Q_t is determined using the formula $Q_t = \frac{6}{\sqrt{3}} \cdot \frac{A_t}{S_t E_t}$, where A_t , S_t , and E_t represent the area, half-perimeter, and length of the longest edge of triangular facet t , respectively. Q_{min} represents the mass of the smallest triangle in the mesh, while Q_{avg} signifies the average mass of all triangles in the mesh. Another important measure for assessing triangle quality is the radius edge ratio $ReR = \frac{R}{e}$, where R is the circumradius and e is the length of the shortest edge of the triangle. Furthermore, ReR_{avg} represents the average radius edge ratio of all triangles in the mesh. In evaluating surface mesh quality, we consider the internal angles of triangular facets, focusing on both the minimum angle θ_{min} and its average $\theta_{min,avg}$. Additionally, we utilize the Hausdorff distance $d_H (\times 10^{-2})$ to quantify the approximation error between the original and generated meshes, and the root mean square distance ($RMS (\times 10^{-3})$) to gauge geometric approximation. The computation time $T(s)$ is meticulously recorded to provide insights into computational performance.

4.1. Remeshing results

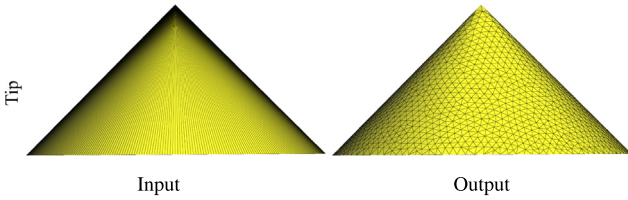
To demonstrate the capability of our method for feature-preservation surface remeshing, we first conduct experiments on several models containing sharp features, including smooth surface, e.g., *Joint*, thin-plate surfaces, e.g., *Disc* and *Finedraw*, complex surfaces, e.g., *Flower* and *Gear*, and organic meshes, e.g., *Horse* and *Lion*. Considering the complexities of different models, the number of sample points varies in this experiment. To be specific, the number of sample points n is set to 3.0k, 5.0k, 7.0k, 9.0k, 20.0k, 3.0k, and 6.0k for the *Disc*, *Joint*, *Finedraw*, *Flower*, *Gear*, *Horse*, and *Lion*, respectively. Fig. 7 presents the remeshing results of our method in various models, and the quality of these triangular meshes are shown in Table 1.

From the results in Fig. 7 and Table 1, we can observe that our method has the capability of producing triangular meshes with sharp feature preservation, and the quality of generated triangular meshes has been improved. Specifically, the results in Fig. 7 prove that our method could detect the sharp features in the original surfaces, and the sample points are successfully projected onto the feature edges to

Table 1

Triangle quality of meshes generated by our method, corresponding to these results in Fig. 7. n = number of vertices, Q_{min} = minimum quality, Q_{avg} = average quality, θ_{min} = minimum angle, $\theta_{min,avg}$ = minimum average angle, $d_H(\times 10^{-2})$ = Hausdorff distance, $RMS(\times 10^{-3})$ = mean square distance.

Model	In/Out	n	Q_{min}	Q_{avg}	ReR_{avg}	θ_{min}	$\theta_{min,avg}$	d_H	RMS
Disc	Input	0.4k	0.218	0.594	1.203	10.090	29.544	–	–
	Output	3.0k	0.471	0.901	0.649	20.128	51.347	0.609	0.633
Joint	Input	0.2k	0.013	0.219	9.151	0.478	9.352	–	–
	Output	5.0k	0.382	0.883	0.688	14.804	50.326	0.542	0.576
Finedraw	Input	7.0k	0.086	0.412	2.369	2.927	18.894	–	–
	Output	7.0k	0.477	0.914	0.635	21.236	52.664	0.195	0.388
Flower	Input	0.9k	0.013	0.574	1.769	0.446	28.701	–	–
	Output	9.0k	0.319	0.875	0.669	15.053	49.665	0.206	0.324
Gear	Input	2.3k	0.000	0.139	26.851	0.013	5.534	–	–
	Output	20.0k	0.329	0.883	0.661	13.767	50.125	0.225	0.167
Horse	Input	21.4k	0.158	0.780	0.771	6.811	42.527	–	–
	Output	3.0k	0.340	0.922	0.630	17.172	53.139	0.569	0.844
Lion	Input	8.3k	0.111	0.598	1.180	4.458	30.405	–	–
	Output	6.0k	0.321	0.907	0.640	16.018	52.029	0.751	0.824

**Fig. 8.** Remeshing result for single cone model. Left: input. Right: result of our method.

preserve the sharp features in the remeshing process. In addition, the quality results in Table 1 further demonstrate the effectiveness and high-efficiency of our method. Compared to the original meshes, our method could generate triangular meshes of higher quality with similar geometric features, that are evaluated by the Hausdorff distance.

Our automated method for detecting sharp features is not perfect. The cusp of a single cone model may have a feature degree of 0, which is caused by the fact that the angle formed by the normals of the adjacent facets with this cusp as one endpoint is too small, as shown in Fig. 8. We can address this issue by employing a novel feature detection method, such as defining the feature degree based on the normal curvature of each vertex. However, automatically identifying sharp features on surface meshes remains a difficult task, as it relies on both the local shape of the model and the global context and semantic information. This inherently makes feature detection a strongly ill-posed problem. Various approaches have been proposed to tackle this challenge [27–29], yet none of them provides a reliable solution for all types of meshes.

4.2. Parameter analysis

The values of the parameters are crucial to the remeshing results in our method. Therefore, we analysis the impact of these parameters on the quality and computational efficiency of generated triangular meshes, including the parameter δ in the projection phase, λ and σ in the gap filling and edge pushing phase, as described in Section 3. In this experiment, we take the *Joint* model as input for our method to generate feature-preserved triangular meshes with different parameter settings. To be specific, the number of sample points are set to $\{1.0k, 5.0k, 10.0k\}$, and the values of the parameters δ , λ , and σ are set to $\{0.1, 0.2, 0.3, 0.4, 0.5\}$, $\{1.0, 1.2, 1.4, 1.6, 1.8\}$, and $\{0.55, 0.65, 0.75, 0.85, 0.95\}$, respectively. Notably, when analysing the impact of one parameter on the remeshing results, other parameters are set to default values. Fig. 9 presents the remeshing results, quality, and computation time of our method with different parameters, which

gives a visualization of the remeshing results for the number of sample points $n = 5.0k$. It should be mentioned that the remeshing result with $n = 5.0k$, $\delta = 0.3$, $\lambda = 1.4$, and $\sigma = 0.75$ are given in Fig. 7.

In terms of the effect of parameter δ on our method, it could be observed from Figs. 7 and 9 that the generated triangular meshes vary with the value of parameter δ . A larger parameter δ means that the radius of the feature edges capturing the projected points is larger. That is, more sample points are projected onto the feature edges and are considered as feature sample points, which speeds up the convergence of our method. However, too large a parameter δ causes too many sample points to be projected onto the feature edges, leading to narrow triangular facets near the feature edges and affecting the quality of generated triangular meshes, as shown in the first column of Fig. 9. Conversely, as the value of parameter δ decreases, fewer sample points are constrained onto the feature edges. Owing to the subsequent feature edge continuity preservation strategy, our method could produce feature-preserved triangular meshes, but at the cost of requiring more time to search near-neighbour sample points for feature preservation.

The quality and feature preservation of the triangular meshes generated by our method is also influenced by the distance between two adjacent feature sample points, which is closely related to the value of parameter λ , as shown in Fig. 7 and the second column of Fig. 9. The larger the value of the parameter λ , the larger the distance between two adjacent feature sample points, and the lesser the number of iterations for generating feature-preserved triangular meshes. However, too large a value of the parameter λ results in a large gap between two adjacent feature sample points, leading to failure feature preservation. In contrast, a smaller value of the parameter λ indicates a smaller gap between two adjacent feature sample points and better feature preservation. Nevertheless, this requires more sample points to be projected onto the feature edges, leading to some narrow triangular facets and more time consumption.

In addition, the parameter σ plays a crucial role in our method by pushing some non-feature sample points away from the feature edges, improving the quality of feature-preserved triangular meshes. Specifically, the parameter σ controls the push-off distance in our method, as described in Section 3. A large value of the parameter σ may cause non-feature sample points to be pushed away from the sharp feature edges, producing poor-quality triangular meshes that are affected by narrow facets. However, a small value of the parameter σ may fail to push non-feature sample points away from sharp feature edges, leading to a failure of feature preservation in remeshing process, as shown in the last column(Top) of Fig. 9.

4.3. Comparison

To further demonstrate the effectiveness and efficiency of our method, we compare against four remeshing techniques, including the

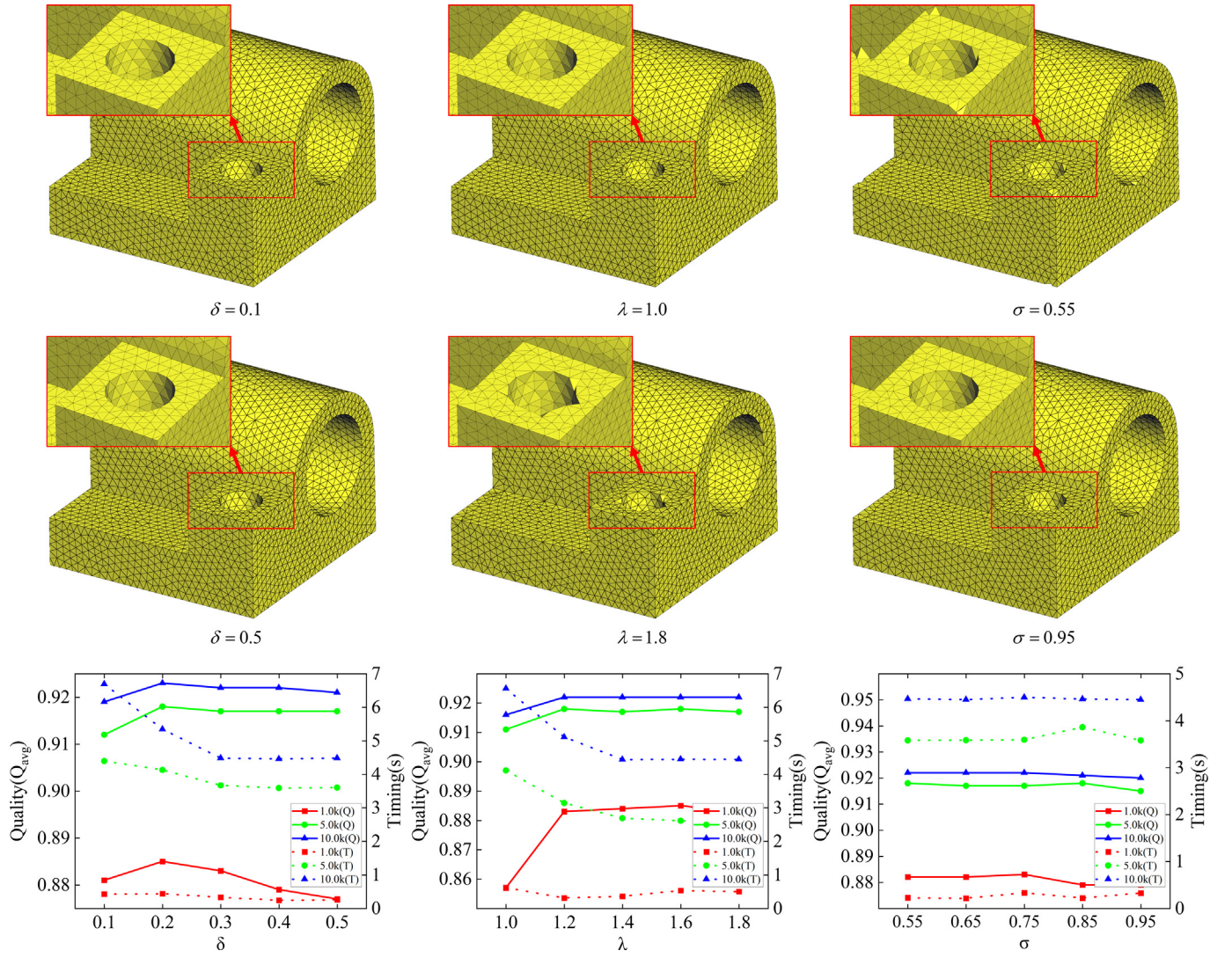


Fig. 9. Visual comparison of the impact of different parameters on the Joint model with 5.0k sample points, where the parameter labels present the detailed values of parameters. From left to right: remeshing results of different values of the parameter δ , λ , and σ , respectively. The impact of parameters δ , λ , σ , and the number of vertices on the running time T and average triangle quality Q_{avg} .

Table 2

The quality and running time of various methods on different models. n = number of vertices, Q_{min} = minimum quality, Q_{avg} = average quality, θ_{min} = minimum angle, $\theta_{min,avg}$ = minimum average angle, $d_H(\times 10^{-2})$ = Hausdorff distance, $RM S(\times 10^{-3})$ = mean square distance, $T(s)$ = running time. The red values indicate the best results of the different types, while the green values indicate the second-best results.

Model	In/Out	n	Q_{min}	Q_{avg}	ReR_{avg}	θ_{min}	$\theta_{min,avg}$	d_H	$RM S$	$T(s)$
Joint	Input	0.2k	0.013	0.219	9.151	0.478	9.352	—	—	—
	RTF [9]	2.0k	0.579	0.917	0.631	27.984	52.939	1.558	7.121	2.786
	MAI [12]	2.0k	0.503	0.761	0.759	31.001	42.250	0.266	0.858	84.805
	VoroCrust [11]	3.8k	0.154	0.731	0.896	5.980	39.339	0.156	0.374	141.067
	CREVO [7]	2.0k	0.730	0.872	0.667	40.012	48.981	0.488	2.346	54.372
	Ours	2.0k	0.349	0.899	0.647	13.176	51.541	0.155	0.337	1.881
Fandisk	Input	7.2k	0.218	0.594	1.223	10.090	29.544	—	—	—
	RTF [9]	2.8k	0.601	0.916	0.632	29.854	52.367	0.756	0.863	2.553
	MAI [12]	2.8k	0.640	0.863	0.665	40.002	49.384	0.195	0.430	56.015
	VoroCrust [11]	3.8k	0.142	0.688	0.995	5.075	36.148	0.455	0.489	45.289
	CREVO [7]	2.8k	0.752	0.879	0.660	40.002	49.790	0.384	1.045	76.933
	Ours	2.8k	0.444	0.902	0.645	21.782	51.687	0.335	0.266	2.051
Crimp	Input	1.6k	0.207	0.317	2.539	7.601	11.928	—	—	—
	RTF [9]	10.0k	0.510	0.908	0.636	25.911	51.387	0.483	1.837	5.278
	MAI [12]	10.0k	0.632	0.858	0.672	35.008	48.705	0.167	0.186	350.699
	VoroCrust [11]	10.0k	0.314	0.629	0.968	20.001	33.829	0.401	0.498	244.117
	CREVO [7]	10.0k	0.750	0.897	0.655	40.001	51.189	0.421	1.845	188.649
	Ours	10.0k	0.427	0.900	0.650	20.312	51.258	0.267	0.109	4.693

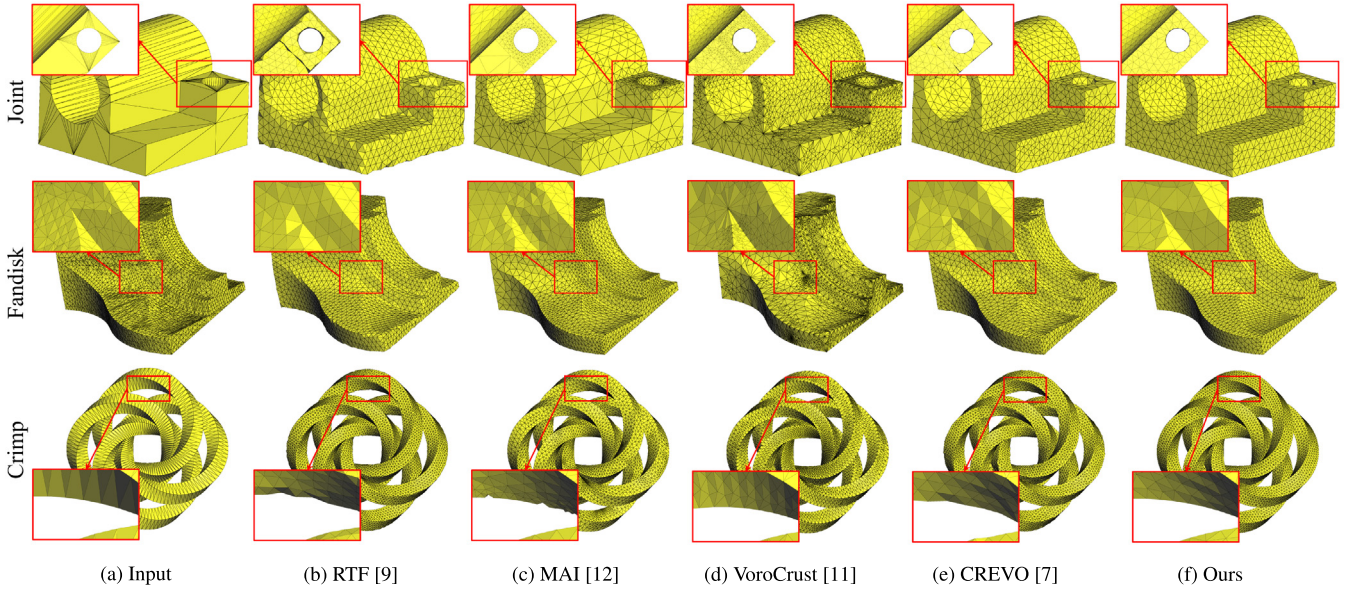


Fig. 10. Comparison results of different methods on various models. From top to bottom: results on the *Joint*, *Fandisk*, and *Crimp* models. From left to right: input models, results of the RTF [9], MAI [12], Vorocrust [11], CREVO [7], and our methods. The quality and computation time of these results are reported in Table 2.

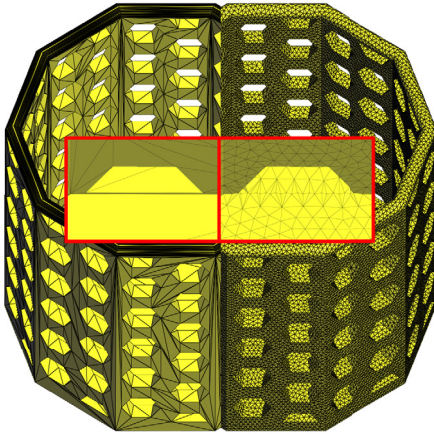


Fig. 11. Limitations of our methodology.

CVT-based algorithm, i.e., RTF [9], Vorocrust [11], as well as feature-preservation algorithms based on local operators, i.e., MAI [12] and CREVO [7]. The RTF [9] as the basic framework of the implementation of our method, approximates the original surface by computing the CVT on the tangent facets of sample points with GPU acceleration. Thus, high-quality triangular meshes could be produced from these optimized sample points. Vorocrust [11] captures the sharp features of the original mesh by introducing paired spherical intersection points within the original mesh. MAI [12] defines a feature intensity function and implicitly achieves sharp feature preservation in remeshing process. Despite the consideration of minimum angle and complexity of generated meshes in MAI [12], we only focus on the sharp feature preservation in this experiment. CREVO [7] considers unique local operator operations for optimization of triangle angles. We use *Joint*, *Fandisk* and *Crimp* as inputs to the various methods, and the number of sample points is set to 2.0k, 2.8k and 10.0k, respectively. The triangle meshes generated by the different methods are shown in Fig. 10. The

quality and computation time of the corresponding results are given in Table 2.

According to these results in Fig. 10 and Table 2, we could observe that our method successfully achieves triangular mesh generation with sharp feature preservation in a variety of models. Although the high-quality triangular meshes produced by RTF [9], fail to handle the sharp features of the original surfaces, as shown in the second column of Fig. 10. Based on the RTF [9] method, our method detects sharp features of the original surfaces and restricts the feature sample points onto the feature edges, thus achieving sharp feature preservation in the remeshing process. However, the restriction of these feature sample points may destroy the CVT structure and result in narrow triangular facets, leading to a slight degradation in the quality of generated triangular meshes. MAI [12] defines the intensity function and detects sharp features based on the curvature of the original surface. Nevertheless, it fails to generate triangular meshes with perfect feature preservation when the input model is too complex, as shown in the *Crimp* model of Fig. 10. Our method detects sharp features of the original surfaces by calculating the angle between adjacent triangular facets, and restricts these feature sample points onto the feature edges, producing triangular meshes with better feature preservation. Particularly, compared to MAI [12], our method achieves better computation efficiency owing to the GPU-accelerated strategy in the remeshing process. Vorocrust [11] utilizes paired spherical intersection seeds to accurately capture the sharp features of the original mesh, resulting in consistent preservation of these features. However, a challenge to be considered is the presence of more narrow triangular facets near the feature edges. Our technique yields superior mesh quality outcomes and preserves the integrity of features, in comparison to Vorocrust [11]. CREVO [7] employs local operator operations to iteratively optimize the minimal corners of the mesh, resulting in a favourable Q_{min} value. However, similar to MAI [12], it is fails to generate a triangular mesh with perfect feature preservation, as shown by the *Crimp* model in Fig. 10.

5. Conclusion

In this paper, we propose a feature-preservation method for surface remeshing. Rather than predetermining feature sample points, we introduce a heuristic strategy to dynamically identify and project feature

sample points onto the original feature edges, achieving feature sample point preservation. Besides, a feature edge continuity preservation strategy is presented to eliminate the large gaps between adjacent feature sample points and edge-close sample points, preserving the reconstructed feature edges. Experimental results on a variety of models indicate the effectiveness and efficiency of our method in yielding high-quality triangular meshes with sharp feature preservation.

Limitations and future work. There are still a few limitations in our work. On the one hand, our proposed feature-preservation method does not apply to non-manifold meshes and FEM meshes. This is primarily due to the presence of multi-face common edges (with three or more faces) in non-manifold meshes, which can adversely affect our feature detection method. And the cell division of the FEM meshes is different from the edge division of our method. On the other hand, when the edges between features are excessively narrow, our method may fail to preserve the features. In our algorithm, the mesh extraction algorithm of RVD [17] is used in the final mesh extraction stage. Since RVD's mesh extraction algorithm is based on pairwise triangulation of Voronoi diagrams, connected Voronoi cells may appear between narrow feature edges, leading to feature edge preservation failure. As shown in Fig. 11. As a result, failure results are generated, which can be solved by using a feature-preservation mesh extraction technique. In the future, the feature-preservation mesh extraction technique is a direction.

CRedit authorship contribution statement

Jingjing Liu: Writing – review & editing, Writing – original draft, Visualization, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Yuyou Yao:** Writing – review & editing, Validation, Project administration, Methodology. **Yue Fei:** Writing – review & editing. **Gaofeng Zhang:** Writing – review & editing. **Liping Zheng:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The models used in this paper are selected from the AIM@Shape and courtesy of the Stanford 3D Scanning Repository. We would like to thank Kai-Mo Hu for their executable programs. This work was supported in part by a grant from the National Natural Science of Foundation of China (No. 62372152) and the National Key Research and Development Plan (No. 2022YFC3900800).

References

- [1] Botsch M, Kobbelt L, Pauly M, Alliez P, Lévy B. Polygon Mesh Processing. CRCPressLLC; 2010.
- [2] Eulitz M, Reiss G. 3D reconstruction of SEM images by use of optical photogrammetry software. *J Struct Biol* 2015;191(2):190–6.
- [3] Chen Y, Ma Z, Zhou H, Zhang W. 3D Print-Scan resilient localized mesh watermarking. In: 2021 IEEE International Workshop on Information Forensics and Security. IEEE; 2021, p. 1–6.
- [4] Liang X, Zhang Y. Hexagon-based all-quadrilateral mesh generation with guaranteed angle bounds. *Comput Methods Appl Mech Engrg* 2011;200(23–24):2005–20.
- [5] Engwirda D, Ivers D. Face-centred Voronoi refinement for surface mesh generation. *Procedia Eng* 2014;82:8–20.
- [6] Wang Y, Yan D-M, Liu X, Tang C, Guo J, Zhang X, et al. Isotropic surface remeshing without large and small angles. *IEEE Trans Vis Comput Graphics* 2018;25(7):2430–42.
- [7] Zhang W-X, Wang Q, Guo J-P, Chai S, Liu L, Fu X-M. Constrained remeshing using evolutionary vertex optimization. *Comput Graph Forum* 2022;41(2):237–47.
- [8] Hurtado J, Montenegro A, Gattass M, Carvalho F, Raposo A. Enveloping CAD models for visualization and interaction in XR applications. *Eng Comput* 2022;1–19.
- [9] Yao Y, Liu J, Wu W, Zhang G, Xu B, Zheng L. Accelerating surface remeshing through GPU-based computation of the restricted tangent face. *Comput Aided Geom Design* 2023;104:102216.
- [10] Jong B-S, Chiang C-H, Lee P-F, Lin T-W. High quality surface remeshing with equilateral triangle grid. *Vis Comput* 2010;26:121–36.
- [11] Abdelkader A, Bajaj CL, Ebeida MS, Mahmoud AH, Mitchell SA, Owens JD, et al. VoroCrust: Voronoi meshing without clipping. *ACM Trans Graph* 2020;39(3):1–16.
- [12] Hu K, Yan D-M, Bommers D, Alliez P, Benes B. Error-bounded and feature preserving surface remeshing with minimal angle improvement. *IEEE Trans Vis Comput Graphics* 2016;23(12):2560–73.
- [13] Alliez P, De Verdière ÉC, Devillers O, Isenburg M. Centroidal Voronoi diagrams for isotropic surface remeshing. *Graph Models* 2005;67(3):204–31.
- [14] Valette S, Chassery JM, Prost R. Generic remeshing of 3D triangular meshes with metric-dependent discrete voronoi diagrams. *IEEE Trans Vis Comput Graphics* 2008;14(2):369–81.
- [15] Yue W, Guo Q, Zhang J, Wang G. 3D triangular mesh optimization in geometry processing for CAD. In: Proceedings of the 2007 ACM Symposium on Solid and Physical Modeling. 2007, p. 23–33.
- [16] Vorsatz J, Rössl C, Kobbelt LP, Seidel H-P. Feature sensitive remeshing. *Comput Graph Forum* 2001;20(3):393–401.
- [17] Yan D-M, Lévy B, Liu Y, Sun F, Wang W. Isotropic remeshing with fast and exact computation of restricted Voronoi diagram. In: *Comput Graph Forum*. 2009;28(5):1445–54.
- [18] Jakob W, Tarini M, Panozzo D, Sorkine-Hornung O, et al. Instant field-aligned meshes. *ACM Trans Graph* 2015;34(6). 189–1.
- [19] Fuhrmann S, Ackermann J, Kalbe T, Goesele M. Direct resampling for isotropic surface remeshing. In: *Vision, Modeling, and Visualization*. 2010, p. 9–16.
- [20] Yan D-M, Wonka P. Gap processing for adaptive maximal Poisson-disk sampling. *ACM Trans Graph* 2013;32(5):1–15.
- [21] Yao Y, Liu J, Fei Y, Wu W, Zhang G, Yan D-M, et al. PowerRTF: Power diagram based restricted tangent face for surface remeshing. *Comput Graph Forum* 2023;42(5):e14897.
- [22] Kazhdan M, Chuang M, Rusinkiewicz S, Hoppe H. Poisson surface reconstruction with envelope constraints. *Comput Graph Forum* 2020;39(5):173–82.
- [23] Sellán S, Jacobson A. Stochastic Poisson surface reconstruction. *ACM Trans Graph* 2022;41(6):1–12.
- [24] Xu M, Xin S, Tu C. An efficient surface remeshing algorithm based on centroidal power diagram. In: Proceedings of the 2019 7th International Conference on Information Technology: IoT and Smart City. 2019, p. 536–42.
- [25] Boltcheva D, Lévy B. Surface reconstruction by computing restricted Voronoi cells in parallel. *Comput Aided Des* 2017;90:123–34.
- [26] Khan D, Plopski A, Fujimoto Y, Kanbara M, Jabeen G, Zhang YJ, et al. Surface remeshing: A systematic literature review of methods and research directions. *IEEE Trans Vis Comput Graphics* 2020;28(3):1680–713.
- [27] Sun Y, Page DL, Paik JK, Koschan A, Abidi MA. Triangle mesh-based edge detection and its application to surface segmentation and adaptive surface smoothing. In: Proceedings. International Conference on Image Processing, vol. 3. IEEE; 2002, p. 825–8.
- [28] Jiao X, Heath MT. Feature detection for surface meshes. In: Proceedings of 8th International Conference on Numerical Grid Generation in Computational Field Simulations. Honolulu HI; 2002, p. 705–14.
- [29] Chen J, Ge X, Wei L-Y, Wang B, Wang Y, Wang H, et al. Bilateral blue noise sampling. *ACM Trans Graph* 2013;32(6):1–11.

Jingjing Liu received the B.S. degree from Soochow University, China, in 2021. He is currently a master candidate in the School of Computer Science and Information Engineering, Hefei University of Technology, China. His research interests include computer graphics and geometric processing. E-mail: 2021111072@mail.hfut.edu.cn

Yuyou Yao received the B.S. and Ph.D. degrees from Hefei University of Technology, China, in 2018 and 2023, respectively. He is currently a lecturer at the School of Public Safety and Emergency Management, Anhui University of Technology, China. His research interests include computer graphics, visualization and geometric processing. E-mail: yaoyy@aust.edu.cn

Yue Fei received the B.S. and M.S. degrees from Hefei University of Technology, China, in 2017 and 2020, respectively. She is currently pursuing the Ph.D. degree in the School of Computer Science and Information Engineering, Hefei University of Technology, China. Her research interests include computer graphics and image processing. E-mail: fy@mail.hfut.edu.cn

Gaofeng Zhang received the B.S. and M.S. degrees from Hefei University of Technology, China, in 2005 and 2008, respectively, and the Ph.D. degree from Swinburne University of Technology, Australia, in 2013. He is currently an associate professor in the School of Software, Hefei University of Technology, China. His research interests include cloud-edge and mobile computing, and computational intelligence. E-mail: g.zhang@hfut.edu.cn

Liping Zheng received the B.S., M.S. and Ph.D. degrees from Hefei University of Technology, China, in 2000, 2004 and 2008, respectively. He is currently a professor in the School of Computer Science and Information Engineering, Hefei University of Technology, China. His research interests include computer graphics, artificial intelligence, and computer simulation. E-mail: zhenglpl@hfut.edu.cn